

Module 8: Website Cloning & Malicious File Delivery

Kayla Teekaram

Overview

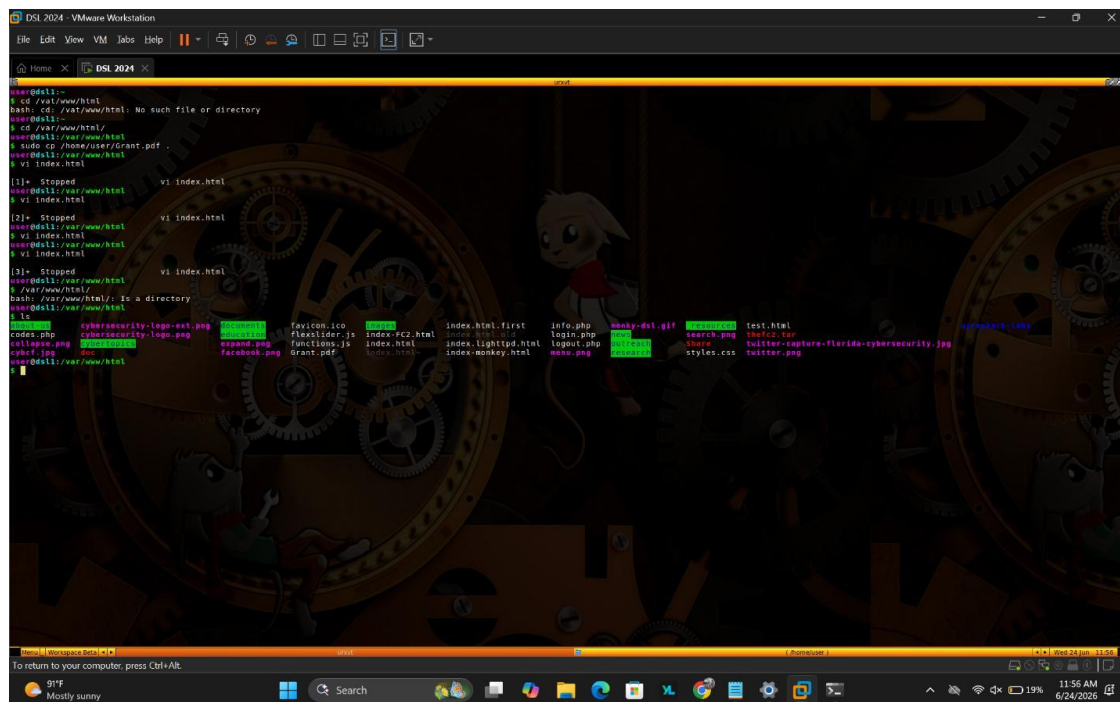
This lab focused on cloning a legitimate organizational website and weaponizing it to deliver a malicious file, simulating how attackers use website spoofing and trojanized documents as part of a social engineering attack chain. The target used as a model was the Florida Center for Cybersecurity (FC²) public site (thefc2.org), and the clone was hosted on a local Apache web server running inside a virtual lab environment (DSL 2024 VM).

Objective

- Stand up a local copy of a real website on an Apache server.
- Modify the cloned site to host and link to an attacker-controlled file.
- Demonstrate what happens when a victim downloads and opens that file.
- Build an understanding of how convincing a spoofed site can look compared to the original, and why users should be cautious about unexpected downloads, even from sites that appear legitimate.

Step 1 — Access the Web Server and Locate the Site Files

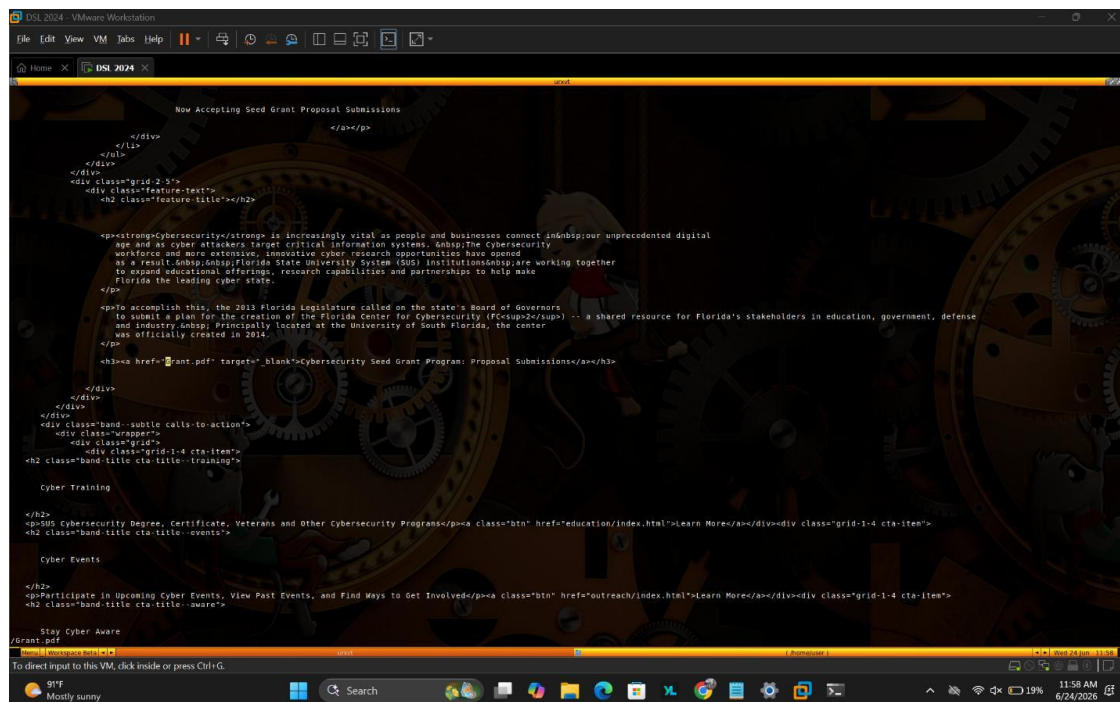
Logged into the lab VM and navigated to the Apache web root with “`cd /var/www/html`”, then ran “`ls`” to list the directory. This revealed the full set of site assets already present (images, JavaScript, PHP scripts, stylesheets, and HTML pages) that make up the cloned Florida Center for Cybersecurity site, including the staged payload file `Grant.pdf` sitting alongside the legitimate content.



Screenshot 1 — Terminal session navigating to /var/www/html and listing the web root's contents, including the staged Grant.pdf payload.

Step 2 — Edit the Site's HTML to Plant the Malicious Link

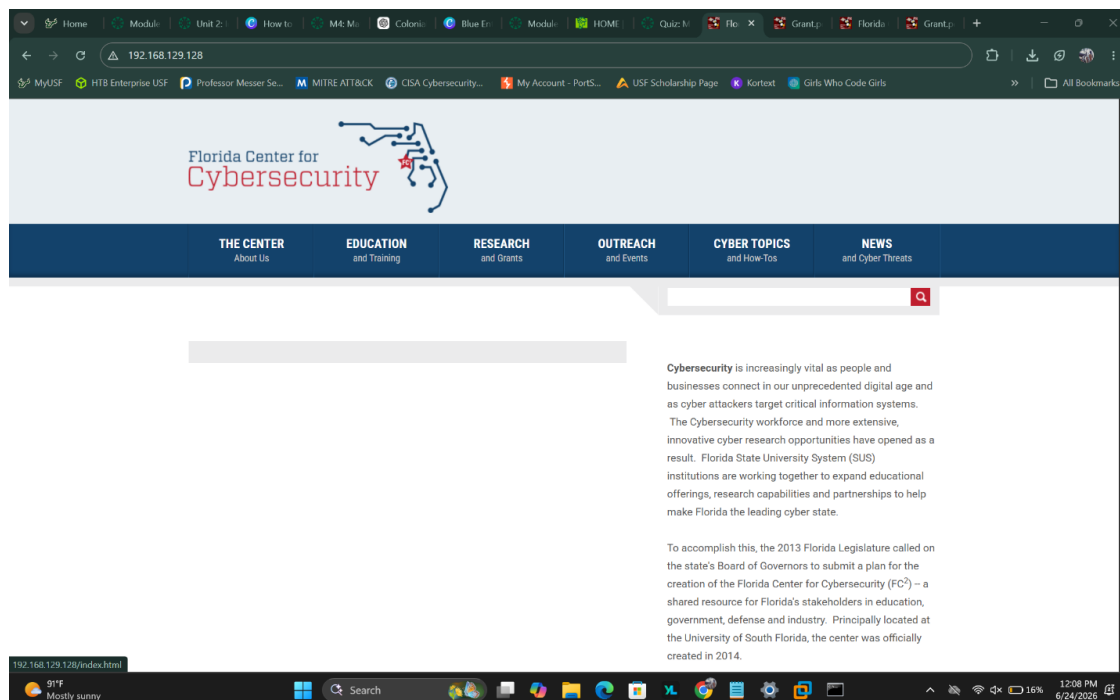
Using vi, the index.html file was opened and edited to insert a link labeled “Cybersecurity Seed Grant Program: Proposal Submissions” that pointed to the staged Grant.pdf instead of a legitimate document. This mirrors a common attacker technique: hiding a malicious payload behind a link that looks like normal, expected content — in this case, a grant proposal document a visitor would have no reason to distrust.



Screenshot 2 — *index.html* opened in vi, showing the injected hyperlink (*href="Grant.pdf"*) behind the “Cybersecurity Seed Grant Program” text.

Step 3 — View the Cloned Site Locally

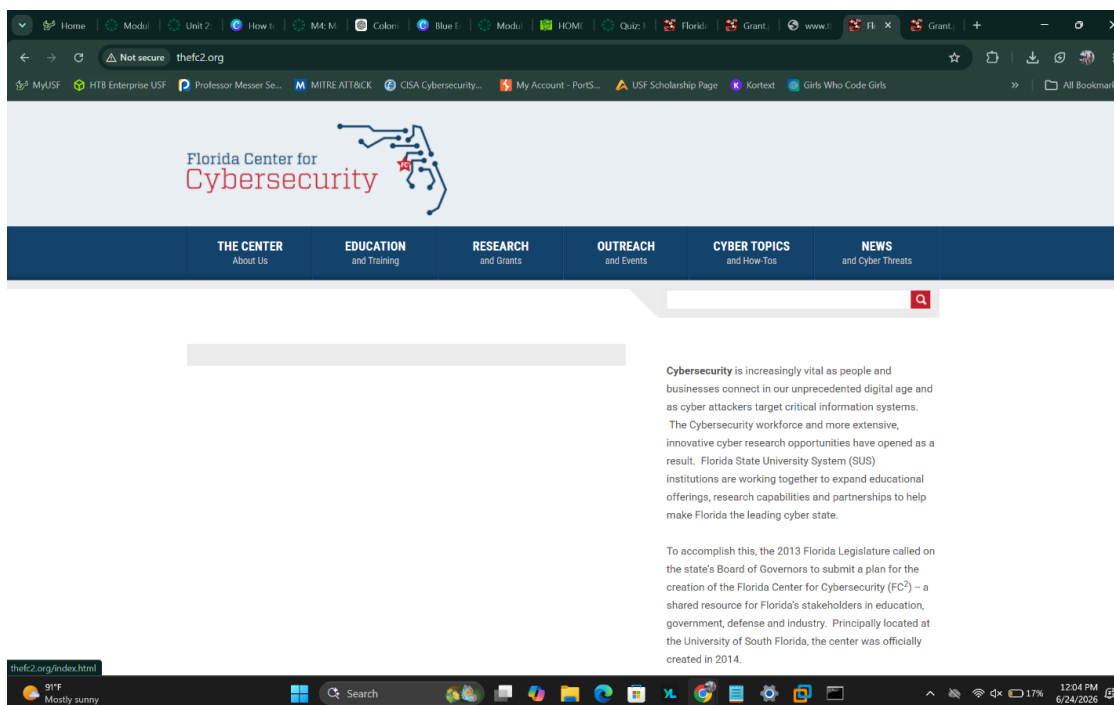
The locally hosted clone was pulled up in a browser at 192.168.129.128 to confirm the edited site rendered correctly, matching the look and layout of the real Florida Center for Cybersecurity homepage.



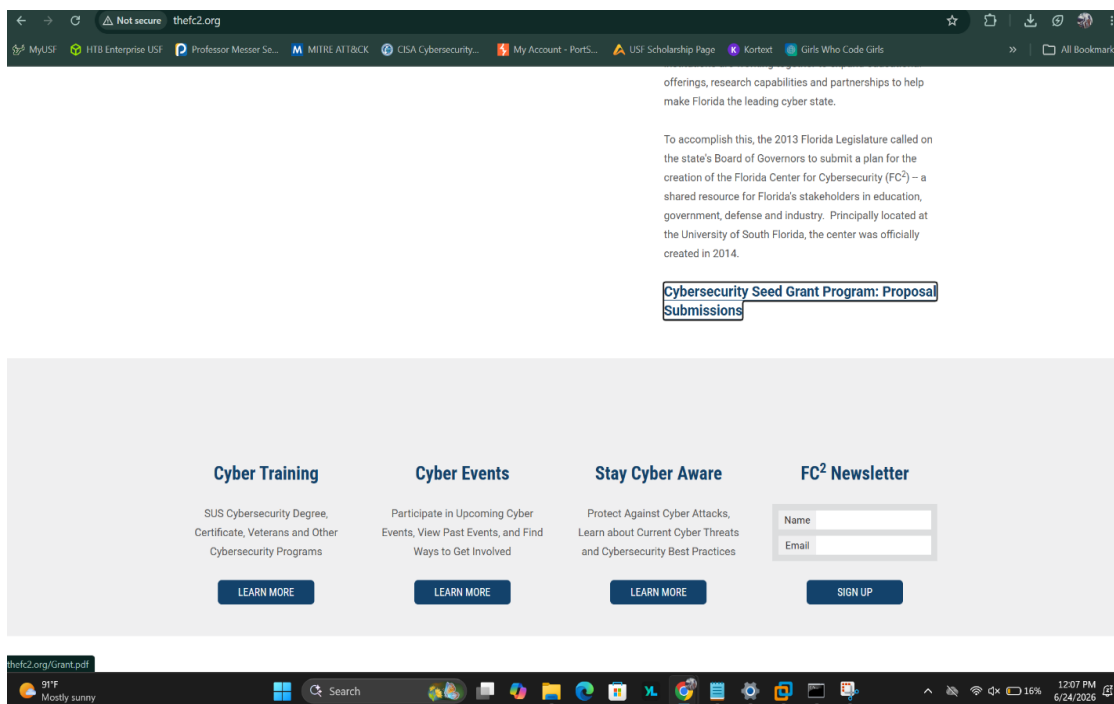
Screenshot 3 — *The cloned site rendering in the browser at 192.168.129.128, visually matching the real FC² homepage.*

Step 4 — Compare the Clone to the Real Site

The real Florida Center for Cybersecurity site (thefc2.org) was opened side by side with the clone for comparison. Visually, the two were nearly identical — same logo, navigation bar, and body copy — illustrating how easy it is to reproduce a trusted site's look and feel well enough to fool a casual visitor. Note the browser's “Not secure” warning in the address bar, a detail users often overlook.



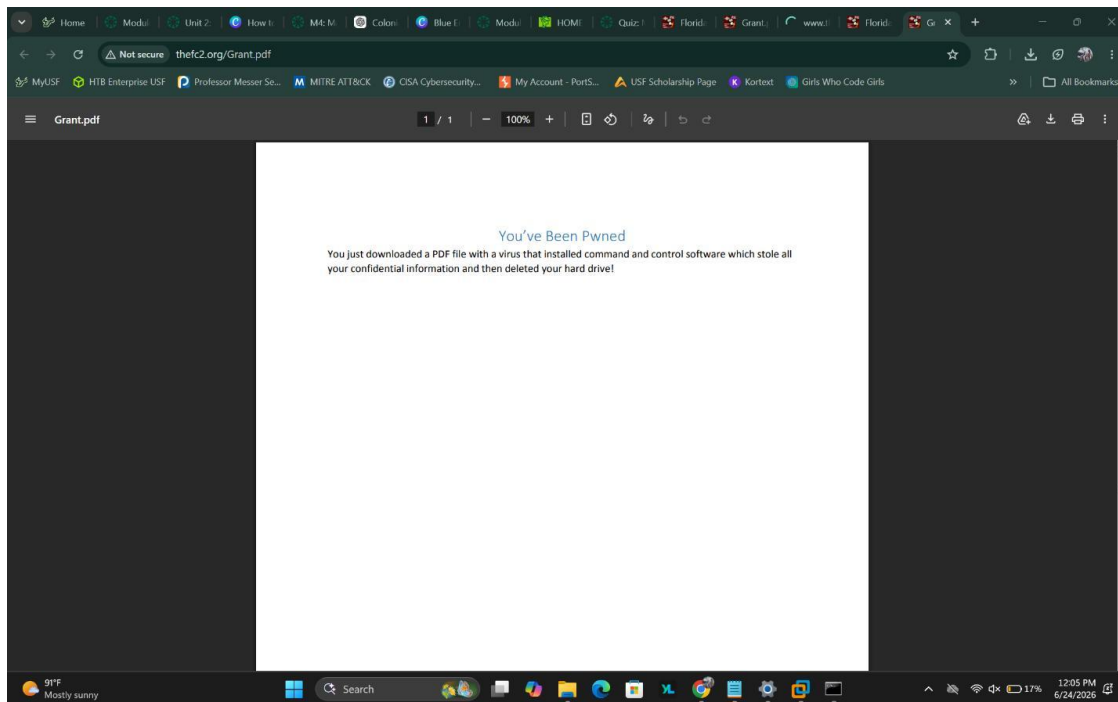
Screenshot 4 — The real thefc2.org site loaded in the browser, shown for visual comparison against the local clone.



Screenshot 5 — Scrolling the real site to locate the “Cybersecurity Seed Grant Program: Proposal Submissions” link, the exact text reused to disguise the malicious link in the clone.

Step 5 — Trigger the Payload

The “Cybersecurity Seed Grant Program: Proposal Submissions” link was clicked, leading to the PDF download. Opening the downloaded Grant.pdf displayed a simulated compromise message: “You’ve Been Pwned — You just downloaded a PDF file with a virus that installed command and control software which stole all your confidential information and then deleted your hard drive!” This confirmed the payload delivery chain worked end-to-end: a user browsing a trustworthy-looking page clicked a plausible link and executed what was framed as a malicious document.



Screenshot 6 — Grant.pdf opened in the browser, displaying the simulated “You’ve Been Pwned” compromise message after download.

Key Takeaways

- Visual trust is not proof of safety. A cloned site can be pixel-for-pixel identical to the original, so URL verification — not visual appearance — is the reliable way to confirm legitimacy.
- Document-based payloads remain effective. Linking a malicious file behind innocuous, context-appropriate text (like a “grant proposal”) increases the odds a target will click without suspicion.
- Web root permissions matter. Being able to drop an arbitrary file into /var/www/html and have it served immediately underscores the importance of locking down who or what can write to a production web server's document root.
- This exercise reinforces why organizations train users to verify links, check sender/hosting legitimacy, and avoid opening unexpected documents — even ones hosted on what looks like a familiar site.